Detecção de Eventos em R com Harbinger

CEFET/RJ

Eduardo Ogasawara

eogasawara@cefet-rj.br | https://eic.cefet-rj.br/~eogasawara

A trilha prática

Começamos com o contrato comum do pacote, inspecionamos dados, visualizamos séries, rodamos detectores e avaliamos resultados — conectando anomalias, change points, suavização, SAX e motifs.

Contrato e setup

Contrato comum do pacote, configuração do ambiente e carregamento dos dados.

Inspeção e visualização

Exploração dos dados, visualização de séries temporais e análise estatística inicial.

Detectores de anomalia

Execução de detectores simples e avaliação de resultados sobre séries reais.

Avaliação hard/soft

Métricas de avaliação hard e soft para medir a qualidade dos detectores.

Change points e transformações

Detecção de change points, suavização e outras transformações de séries temporais.

SAX, motifs e discords

Representação simbólica SAX, descoberta de motifs e identificação de discords.

Workflow recorrente

Carregar dados.

Escolher ou configurar um método.

Executar detecção.

Inspecionar estrutura de saída.

Plotar e avaliar contra rótulos quando existirem.

ENTRADA: série temporal dataset$serie (e opcionalmente dataset$event)
PRÉ-CONDIÇÃO: pacote harbinger carregado; dados sem NA

PASSO 1 — Instanciar o método: model <- metodo(parametros)
PASSO 2 — Ajustar (quando necessário): model <- fit(model, dataset$serie)
PASSO 3 — Detectar: detection <- detect(model, dataset$serie)
PASSO 4 — Visualizar: har_plot(model, dataset$serie, detection, dataset$event)
PASSO 5 — Avaliar (quando há rótulos): evaluate(model, detection$event, dataset$event)

SAÍDA: data.frame com colunas idx, event, type (e atributos extras por método)
PÓS-CONDIÇÃO: detection$event é vetor lógico alinhado ao índice da série

Setup mínimo

Os exemplos usam uma semente comum para reprodutibilidade.

O pacote `harbinger` fornece detectores, avaliação e visualização.

`daltoolbox` aparece em exemplos que usam transformações e modelos auxiliares.

```
source(url("https://raw.githubusercontent.com/cefet-rj-dal/harbinger/master/examples/seed.R"))
library(harbinger)
library(daltoolbox)
```

Inspeção de dados: conheça antes de detectar

A escolha do detector depende da coleção: tamanho, canais, rótulos e estrutura. Inspecionar antes evita escolher algoritmo apenas pelo nome.



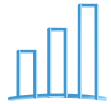
Estrutura da coleção

Tamanho, canais, rótulos e tipo (univariada ou multivariada).



Metadados separados

idx, event, type, seq e seqlen são separados dos sinais pelo helper.



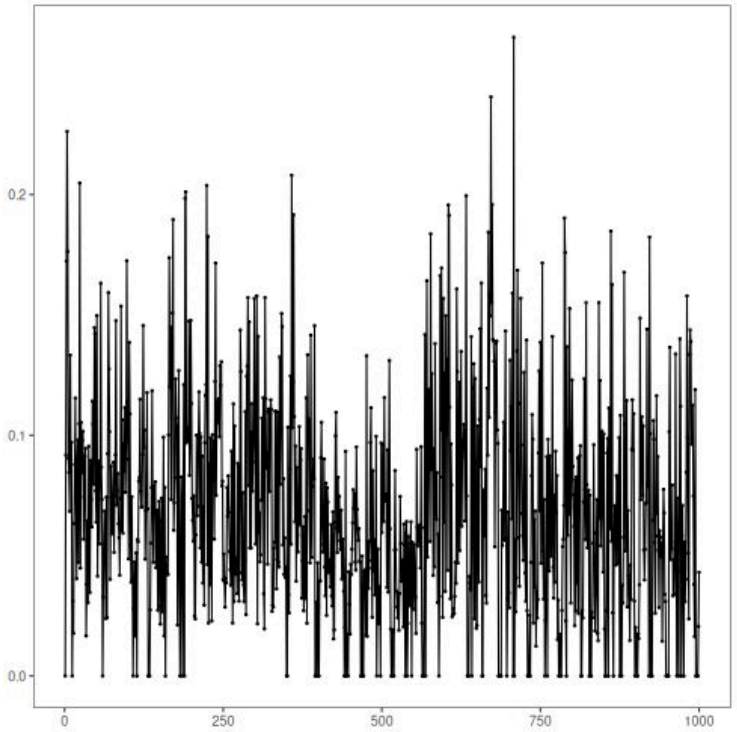
Benchmark real

O tutorial usa o A1Benchmark como exemplo de coleção real.

```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
info <- dataset_summary(A1Benchmark)
```

Benchmark Yahoo A1

Inspeção visual da primeira série de uma coleção rotulada.




```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
a1_info <- show_dataset(A1Benchmark, "A1Benchmark")
## Dataset: A1Benchmark
## Number of series: 67
## Dataset type: univariate
## Signals in the first series: value
## Column plotted with har_plot(): value
## Plot preview length: 1000 observations
plot_dataset_preview(a1_info)
```

[harbinger/examples/datasets/01-benchmarks-yahoo_benchmarks.md at master · cefet-rj-dal/harbinger]

Séries univariadas e multivariadas

Univariada

Uma coluna de sinal.

Entrada direta para muitos detectores.

Boa para iniciar o workflow.

Multivariada

Várias colunas de sinal.

Requer escolher canais ou combinar dimensões.

A inspeção orienta a escolha.

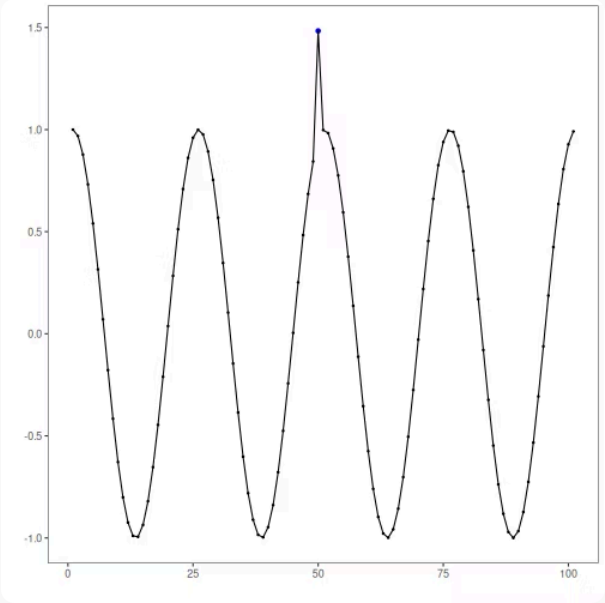
Dados rotulados para o primeiro contato

O primeiro exemplo usa uma série pequena com rótulo de evento.

Isso permite entender entrada, saída e visualização sem discutir ainda um detector específico.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)
```

Visualização inicial e diagnóstico



Antes de detectar, visualizamos a série e os eventos conhecidos.

har_plot() como diagnóstico

Mais que estética

Revela tendência, bursts, mudança de nível e ruído antes de qualquer ajuste.

Primeira checagem

Use antes de ajustar janelas, limiares e detectores.

Eventos sobrepostos

Rótulos do ground truth aparecem junto com a detecção para comparação visual imediata.

Contrato comum do Harbinger

`harbinger()` representa o contrato comum do pacote.

Um método recebe uma série e retorna uma saída padronizada orientada a eventos.

Esse contrato facilita trocar detectores sem trocar todo o workflow.

Eventos aparecem quando usamos detectores concretos, como `hanr_arima()`, `hcp_amoc()` ou `hmo_sax()`.

```
ENTRADA: série temporal dataset$serie
PRÉ-CONDIÇÃO: método instanciado via harbinger()

PASSO 1 — Instanciar:  model <- harbinger()
PASSO 2 — Detectar:    detection <- detect(model, dataset$serie)
PASSO 3 — Inspecionar: head(detection)
      # idx | event | type
      #   1 | FALSE |
      #   2 | FALSE |
      #   3 | FALSE |

SAÍDA: data.frame padronizado com idx, event, type
PÓS-CONDIÇÃO: interface idêntica para qualquer detector do
pacote
```

Saída padronizada

Estrutura mínima

A saída sempre contém `idx` (índice), `event` (TRUE/FALSE) e `type` (tipo do evento). Detectores podem adicionar atributos extras.

Intercambiabilidade

A padronização permite trocar detectores sem mudar o código de avaliação e plotagem — a mesma interface funciona para qualquer detector do pacote.

```
idx | event | type
  1 | FALSE |
  2 | FALSE |
 42 | TRUE  | anomaly
```

Primeiro detector de anomalia

`hanr_histogram()` é um baseline simples e interpretável.

A ideia é tratar valores em bins raros como anômalos.

Ele não modela dinâmica temporal, mas é útil como referência inicial.

Baseline simples

Não modela dinâmica temporal. Útil como referência inicial.

Parâmetro único

`density_threshold` controla o limiar de raridade dos bins.

Interpretável

Fácil de explicar: valores em bins raros são anômalos.

Histograma: configurar e detectar

O histograma divide a série em **bins de frequência** e marca como anomalia os pontos que caem em bins com densidade muito baixa.

Parâmetro principal: `density_threshold` — limiar de densidade abaixo do qual um ponto é considerado raro.

Use como **baseline interpretável** quando não há necessidade de modelagem temporal. Simples, rápido e transparente.

```
model <- hanr_histogram(density_threshold = 0.05)
detection <- detect(model, dataset$serie)
head(detection)
```

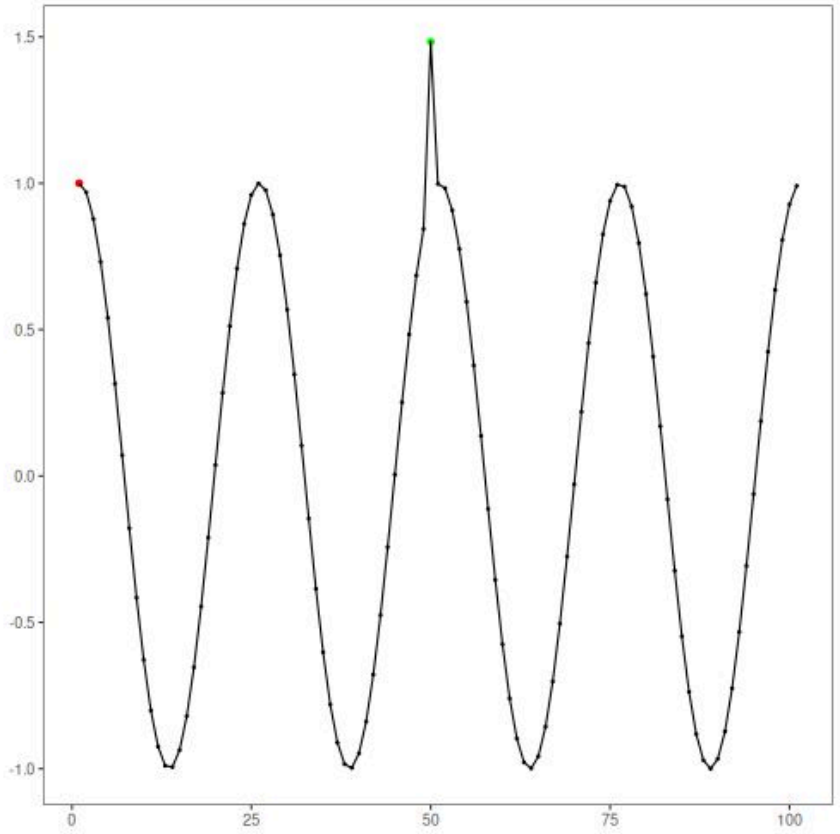
```
ENTRADA: série univariada dataset$serie
PRÉ-CONDIÇÃO: série numérica sem NA

PASSO 1 — Construir histograma de frequências
          da série
PASSO 2 — Calcular densidade de cada bin
PASSO 3 — Marcar como anomalia pontos em bins
          com densidade < density_threshold
PASSO 4 — Retornar índices marcados como
          event = TRUE

SAÍDA: detection com colunas idx, event
       (TRUE/FALSE), type = "anomaly"
PÓS-CONDIÇÃO: pontos em bins com densidade abaixo de density_threshold
              são marcados como anomaly; a proporção final de anomalias
              depende da distribuição e dos bins
```

Resultado do detector por histograma

Detecções do baseline sobre a série rotulada.




```r
library(daltoolbox)
library(harbinger)
# Load example anomaly datasets
data(examples_anomalies)
# Select a simple anomaly dataset
dataset <- examples_anomalies$simple
head(dataset)
har_plot(harbinger(), dataset$serie, event = dataset$event)
model <- hanr_histogram(density_threshold = 0.05)
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
print(detection |> dplyr::filter(event == TRUE))
evaluation <- evaluate(model, detection$event, dataset$event)
print(evaluation$confMatrix)
har_plot(model, dataset$serie, detection, dataset$event)
```

Avaliação do baseline: hard e soft

Quando há ground truth, a detecção pode ser comparada com os rótulos. A mesma detecção pode ser lida por avaliadores diferentes — isso separa o resultado do detector do critério de julgamento.

```
# Avaliação direta
evaluation <- evaluate(model, detection$event, dataset$event)
evaluation$confMatrix

# Comparando avaliadores
hard_result <- evaluate(har_eval(), detection$event, dataset$event)
soft_result <- evaluate(har_eval_soft(), detection$event, dataset$event)
```

O que retorna

Matriz de confusão (TP, FP, FN, TN) e métricas derivadas: precision, recall, F1.

Por que comparar

Hard e soft medem a mesma detecção com critérios diferentes. F1 soft ≥ F1 hard sempre.

Avaliação estrita e avaliação suave

Hard evaluation

Exige coincidência direta.

Útil para rótulos pontuais bem definidos.

`har_eval()`

Soft evaluation

Aceita desalinhamento temporal pequeno.

Mais adequada a eventos com imprecisão temporal.

`har_eval_soft()`

ENTRADA: detection$event (vetor lógico detectado), dataset$event (ground truth)
PRÉ-CONDIÇÃO: vetores de mesmo comprimento; alinhados por índice

PASSO 1 — Hard evaluation (har_eval):
    Comparar detection$event == dataset$event ponto a ponto
    Calcular TP, FP, FN, TN diretos
PASSO 2 — Soft evaluation (har_eval_soft):
    Para cada evento detectado, verificar se há rótulo
    dentro de uma janela de tolerância temporal
    Calcular TP, FP, FN com margem de desalinhamento
PASSO 3 — Calcular métricas: precision, recall, F1

SAÍDA: lista com confMatrix, precision, recall, F1
PÓS-CONDIÇÃO: har_eval_soft atribui crédito parcial a detecções próximas no tempo; os valores dependem da janela sw_size e das associações entre eventos e detecções

Anomalia baseada em modelo

`hanr_arima()` modela o comportamento esperado da série.

Anomalias aparecem como resíduos grandes.

Esse exemplo conecta o Harbinger à teoria de modelos temporais.

Modelagem temporal

ARIMA captura autocorrelação e tendência da série.

Resíduos como sinal

Anomalias aparecem onde o modelo erra mais do que o esperado.

Mesmo workflow

`fit()`, `detect()` e `evaluate()` idênticos ao histograma.

ARIMA: ajustar, detectar e avaliar

Sobre o método

Modela o comportamento esperado da série temporal. Anomalias são identificadas como **resíduos grandes** em relação ao modelo ajustado.

Parâmetros principais

Ordem `p,d,q` selecionada automaticamente pelo detector.

Quando usar

Séries estacionárias ou com tendência e diferenciáveis; sem valores `NA`.

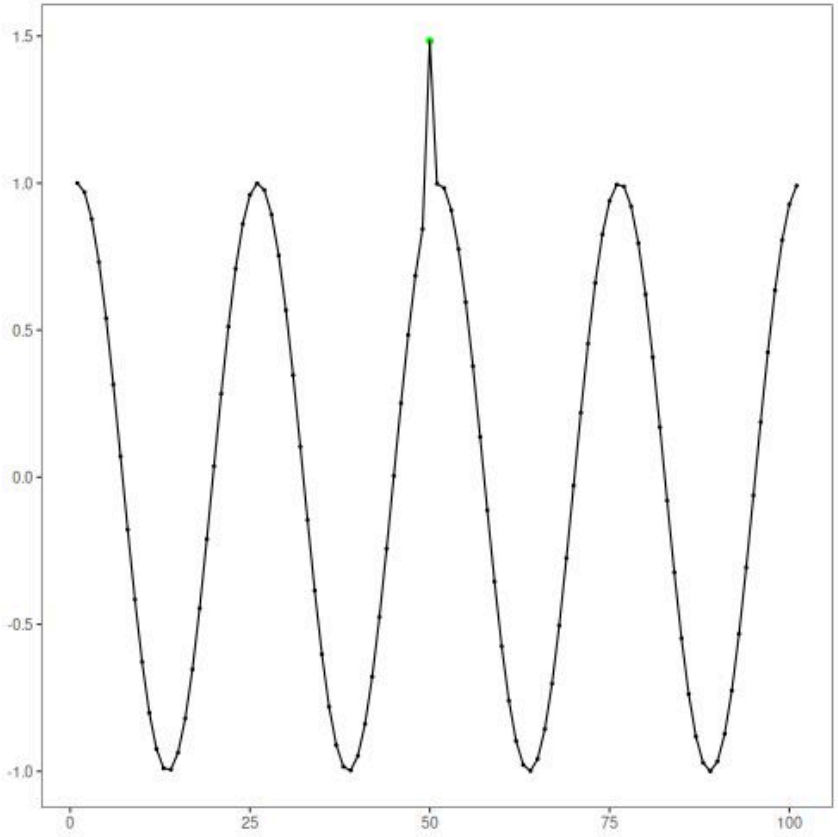
```
model <- hanr_arima()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
        evaluate(model, detection$event,
dataset$event)
```

```
ENTRADA: série temporal dataset$serie
PRÉ-CONDIÇÃO: série numérica sem NA após tratamento interno

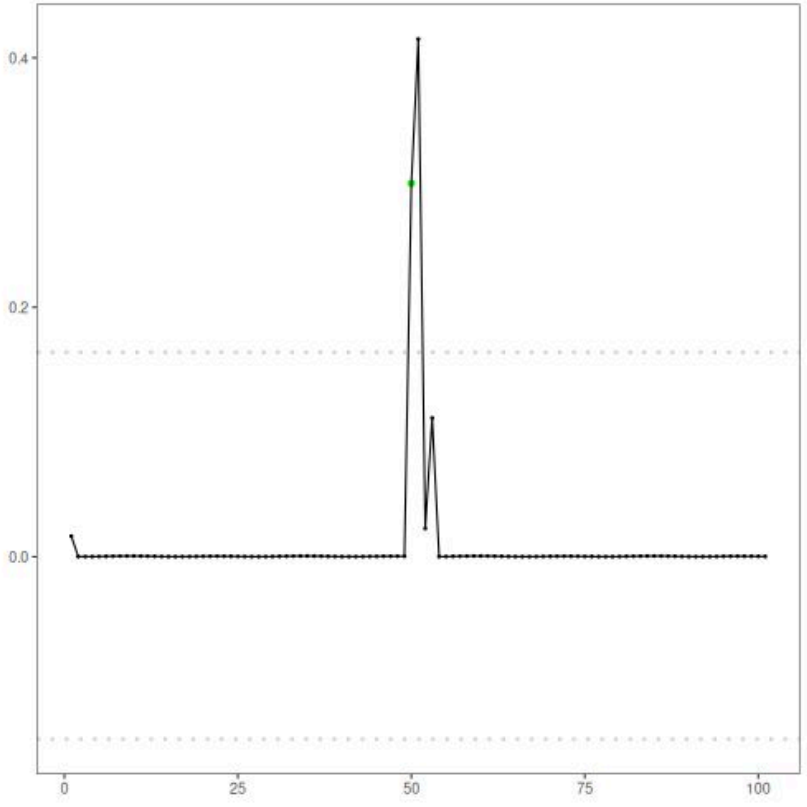
PASSO 1 — Ajustar modelo:
        model <- fit(hanr_arima(), dataset$serie)
        ordem (p,d,q) selecionada por auto.arima
PASSO 2 — Detectar:
        reajustar ARIMA com ordem armazenada
        calcular resíduos e_t
PASSO 3 — Converter resíduos em distância:
        score_t <- har_distance(e_t)
PASSO 4 — Aplicar filtro de outliers:
        candidatos <- har_outliers(score_t)
        candidatos <- har_outliers_check(candidatos, score_t, serie)
PASSO 5 — Ignorar janela inicial sw_size
        e restaurar índices originais

SAÍDA: detection com idx, event, type = "anomaly"
PÓS-CONDIÇÃO: limiar e candidatos seguem os utilitários internos do Harbinger,
            não um k fixo universal
```

Detecção por ARIMA



O modelo detecta anomalias a partir do desvio em relação ao comportamento esperado.



O resíduo explica por que o detector marcou determinados pontos.

Primeiro detector de change point

- `hcp_amoc()` procura uma mudança dominante.
- O alvo não é um ponto isolado raro, mas uma transição de regime.
- É uma entrada simples antes de métodos de múltiplas mudanças.

At Most One Change

Detecta no máximo uma mudança dominante de regime.

Mudança de nível

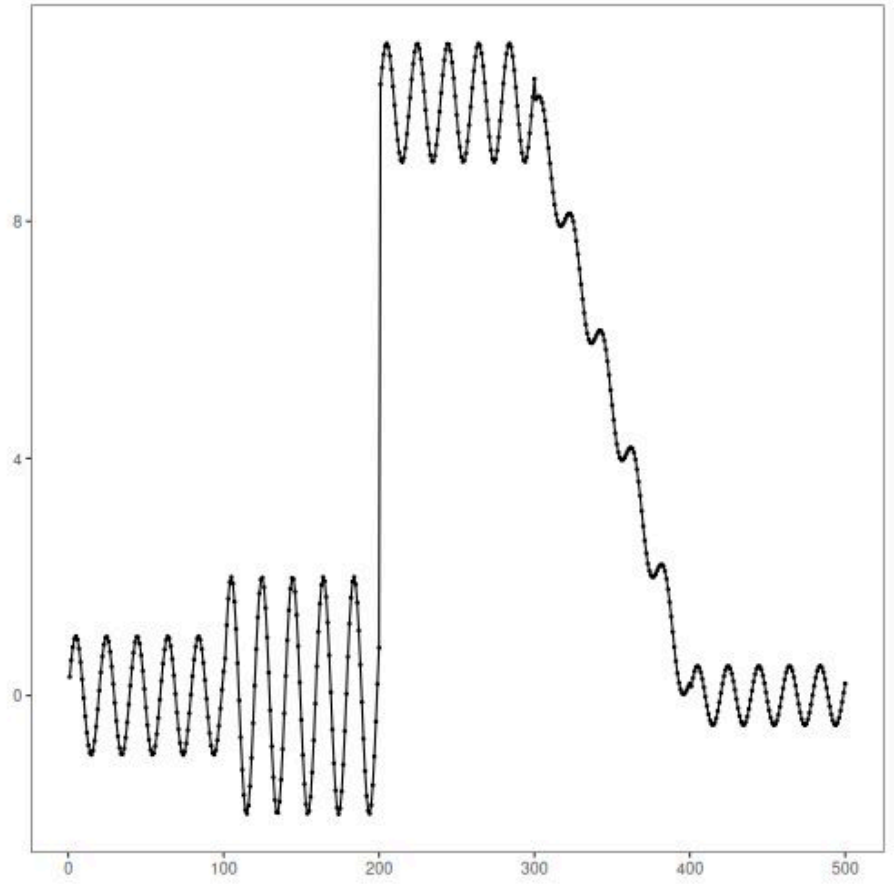
Identifica transição entre dois estados distintos da série.

Ponto de partida

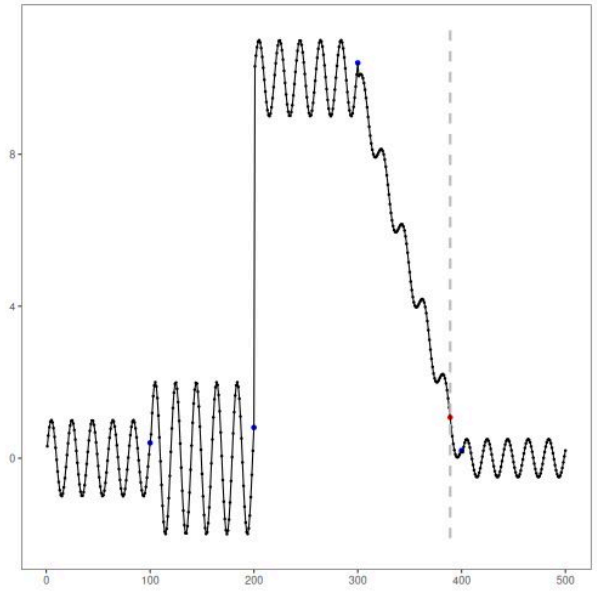
Entrada simples antes de métodos de múltiplas mudanças como PELT ou BinSeg.

Série com change point rotulado

A visualização mostra a série antes de aplicar AMOC.



AMOC no Harbinger



O que é AMOC?

At Most One Change — detecta uma única mudança dominante de regime na série temporal.

Quando usar

Ideal para séries com **uma transição clara** entre dois estados distintos, sem múltiplas rupturas.

Resultado

O algoritmo usa `cpt.meanvar` com penalização MBIC e pode não marcar nenhum evento se o teste não sustentar a mudança.

```
model <- hcp_amoc()
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie, detection,
dataset$event)
```

ENTRADA: série temporal dataset$serie
PRÉ-CONDIÇÃO: série numérica sem NA; espera-se no máximo uma mudança dominante

PASSO 1 — Normalizar índices e remover NA internamente
PASSO 2 — Executar:
cpt.meanvar(serie, method="AMOC",
penalty="MBIC", test.stat="Normal")
PASSO 3 — Extrair change point retornado pelo objeto cpt,
excluindo o marcador final da série quando existir
PASSO 4 — Marcar índices extraídos como event = TRUE,
type = "changepoint"

SAÍDA: detection com idx, event, type
PÓS-CONDIÇÃO: detecta no máximo um change point relevante;
pode não marcar evento se o teste/penalização não sustentar mudança

O gráfico ao lado mostra o changepoint detectado pelo AMOC sobre a série rotulada.

Transformações importam

- Antes de detectar, podemos transformar a série.
- Suavização revela padrões de baixa frequência e reduz variação local.
- A escolha da transformação muda o que o detector enxerga.

Moving average smoother

- `mas()` calcula média móvel.
- Janelas menores preservam detalhe.
- Janelas maiores produzem sinal mais suave.

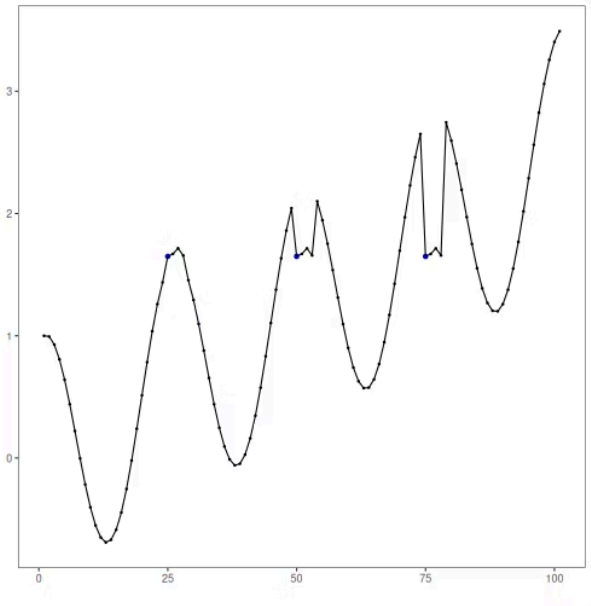
```
ENTRADA: série temporal x de tamanho n; ordem k = order
PRÉ-CONDIÇÃO: série numérica; 1 <= k <= n

PASSO 1 — Para t de 1 até n-k+1:
        xt[t] <- média(x[t], x[t+1], ..., x[t+k-1])
PASSO 2 — Converter xt para objeto ts
PASSO 3 — Guardar atributos:
        order = k
        xi = primeiros k-1 valores de x
        xf = últimos k-1 valores de x

SAÍDA: série suavizada de tamanho n-k+1
PÓS-CONDIÇÃO: a série resultante fica mais curta;
            rótulos/eventos precisam ser realinhados ao usar mas()
```

```
ma_5  <- mas(dataset$serie, order = 5)
ma_15 <- mas(dataset$serie, order = 15)
```

SAX e XSAX no workflow



Discretização Simbólica

Converte séries temporais contínuas em sequências de símbolos discretos, permitindo análise simbólica eficiente.

- **SAX**: usa um alfabeto compacto de tamanho `alpha`
- **XSAX**: permite codificação mais rica com alfabeto alfanumérico de até 36 símbolos (0-9A-Z)

Parâmetro Principal

`alpha` = tamanho do alfabeto (ex: 8 para SAX, 16 para XSAX)

Quando Usar

- Pré-processamento para **motif discovery**
- Compressão de séries temporais
- Redução de dimensionalidade simbólica

ENTRADA: série temporal dataset$serie; alpha
PRÉ-CONDIÇÃO: série numérica; SAX: 2 <= alpha <= 26;
XSAX: 2 <= alpha <= 36

PASSO 1 — Normalizar série por z-score
PASSO 2 — Calcular quantis da própria série normalizada
para dividir os valores em alpha bins
PASSO 3 — Atribuir cada ponto ao seu bin
PASSO 4 — Mapear bins para códigos simbólicos:
SAX usa A-Z
XSAX usa 0-9A-Z

SAÍDA: vetor simbólico com um símbolo por observação
PÓS-CONDIÇÃO: discretização preserva o comprimento da série;
janelas simbólicas são criadas depois, por detectores como hmo_sax()

Motifs e discords

A trilha fecha com análise de padrões recorrentes.

Motif discovery pergunta quais subsequências se repetem.

Discords são subsequências isoladas em relação ao restante.

Motifs

Subsequências que se repetem com frequência ≥ suporte mínimo. Indicam padrões recorrentes no comportamento da série.

Marcados como `type = "motif"`.

Discords

Subsequências mais distantes do vizinho mais próximo. Indicam comportamentos raros ou anômalos.

Marcados como `type = "discord"`.

Detector simbólico de motifs

O que é hmo_sax()?

Recebe a série numérica diretamente e aplica trans_sax() internamente antes de buscar padrões recorrentes.

- **Motifs:** subsequências que se repetem com frequência ≥ qtd
- **Discords:** exigem detector específico, como hdis_sax()

Parâmetros principais

- a: tamanho do alfabeto (1 a 26)
- w: tamanho da janela simbólica
- qtd: suporte mínimo (≥ 2)

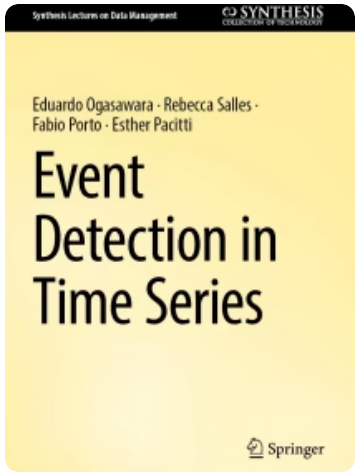
```
model <- hmo_sax(8, 15, 3)
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
```

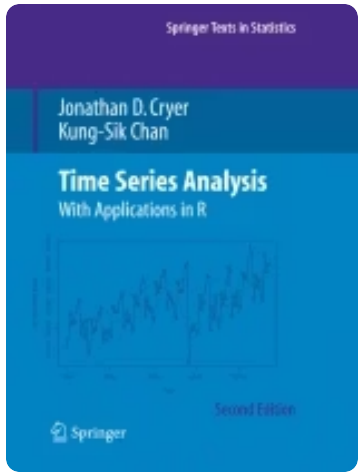
```
ENTRADA: série numérica dataset$serie; alfabeto a; janela w; suporte qtd
PRÉ-CONDIÇÃO: a entre 1 e 26; w >= 1; qtd >= 2

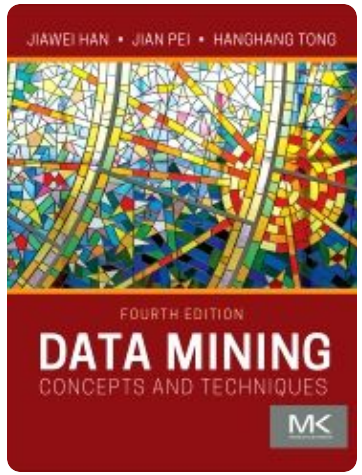
PASSO 1 — Aplicar trans_sax(a) internamente sobre dataset$serie
PASSO 2 — Criar janelas simbólicas com tspredit::ts_data(tss, w)
PASSO 3 — Concatenar símbolos de cada janela para formar sequências
PASSO 4 — Agrupar sequências iguais e manter grupos com total_count >= qtd
PASSO 5 — Remover motifs triviais próximos usando faixa em torno de w/2
PASSO 6 — Marcar primeiras ocorrências válidas como event = TRUE,
          type = "motif", com colunas seq e seqlen

SAÍDA: detection com idx, event, type = "motif", seq, seqlen
PÓS-CONDIÇÃO: hmo_sax detecta motifs recorrentes;
              discords exigem detector específico
```

Referências Bibliográficas







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4ª ed., 2022.